1. HDFS的读写流程是什么?

   读文件：

   1. 客户端向Namenode发起读文件的请求，然后Namenode返回存在数据的DataNode的地址。
   2. 客户端根据就近原则向Datanode读取数据。
   3. Datanode以packet为单位向客户端传输数据。

   写文件：

   1. 客户端向Namenode发起写文件的请求，NameNode查询父文件是否存在等信息。
   2. 如果存在，Namenode返回写入的Datanode地址。
   3. 客户端在各个DataNode之间建立一条通信管道，以Packet为单位进行传输。

2. MapReduce的执行流程是什么（Split，Map，shuffle，Reduce）？

   Split阶段：根据blocksize来切分文件，其中每一个文件对应一个map task。

   Map阶段：根据每一个maptask任务来切分成键值对的形式。

   Shuffle阶段：

   - map端shuffle：KV键值对被放入环形缓冲区中，当环形缓冲区达到80%的阈值时对里面的数据进行快排生成小文件。当小文件达到一定数量之后通过归并排序合并成一个有序的大文件。
   - reduce端shuffle：根据分区从Map端拉取数据，然后使用归并排序将不同的map拉取的数据进行归并成有序的大文件。

   Reduce阶段：根据最终有序的大文件得到最终结果。

3. Yarn由哪几个部分组成，常见的资源调度器有哪些?

   Yarn主要由ResourceManager，ApplicationMaster，NodeManager和Container组成

   有三种常见的调度器：

   - 先进先出：根据顺序来执行任务。
   - 容量调度器：规定每个队列的最小容量，这样会保证小任务都能够得到资源来运行。比如为每一个部门来设置一个容量大小。
   - 公平调度器：为每一个队列来设置权重，保证重要的任务能够优先执行。

4. Hive任务的转化流程

   Hive的本质就是将hivesql转换成一个MR任务然后交给Yarn进行调度。流程如下：

   1. 通过Drive来生成语法树。
   2. 检查语法树是否出现问题。
   3. 根据语法树来生成逻辑执行计划。
   4. 通过元数据信息来优化逻辑执行计划。
   5. 将逻辑执行计划转换成物理执行计划，生成最终的MR任务。

5. Hive中常见的调优方法有哪些?

   Hive调优可以分为sql调优和参数调优：

   sql调优：

   列裁剪；Where提前过滤；谓词下推

   count(distinct)转换成count+group by；order by转换成distribute by+sort by。

   小表关联小表；使用相同桶倍数的分桶表；skew-join（只支持inner join）

参数优化：

调整map和reduce的核数和内存大小

控制map个数

控制reduce的个数：通过控制每个reduce处理的数量大小或者直接进行指定。

group by开启map端聚合。

6. 你是怎么判断是否出现了数据倾斜现象？怎么解决常见的数据倾斜问题（map端和reduce端）？

   如果一个任务进度卡在99%，并且我们发现yarn日志中不同task的shuffle数据量大小差别很大。来判断是否出现了数据倾斜问题。

   常见的数据倾斜有两个：
   map端的数据倾斜：

   由于数据大小分布不均且文件不可拆分导致出现数据map端出现数据倾斜。可以通过map端小文件合并或者修改输入文件格式解决。

   Reduce端的数据倾斜：

   在这之前都可以通过列裁剪和提前聚合来减少数据量。

   - group by导致的数据倾斜：针对热点值进行加盐处理，之后再去盐进行二次聚合得到最终结果。
   - join导致的数据倾斜：如果维表大小不大，可以直接对维表进行膨胀数倍进行加盐join；如果维表较大，则单独提取出热点键，然后将维表膨胀数倍之后进行加盐join。

7. 怎么进行小文件的治理（预防和对已有小文件的处理）？

   小文件的预防：

   hive生成的文件个数等于reducetask的个数，我们可以通过修改每个reduce处理的数据量大小来控制最终生成的文件个数。

   Spark：spark生成文件的个数等于最终partition的个数，我们可以通过repartition来指定最终生成文件的数量。

8. 如果你遇到一个任务执行时间突然变慢，你会怎么排查问题？

   首先对比历史版本信息，查看代码是否发生了修改，如果发生修改检查一下修改的内容是否可能导致任务的延时，比如修改了任务的并行度；变更了口径等。

   然后查看任务运行日志和之前的进行对比

   如果整体的job/stage都变慢了，可以从以下两个方面来排查问题：

   - 资源不足导致并行度不够：查看task的lauchtime是否差的很多，如果差的很多说明资源不足导致并行度不够。
   - 数据膨胀导致task处理的数据量过大：通过yarn查看每个task处理的数据量大小是否明显变大，如果是的话说明源头表发生了数据膨胀。

   如果部分job/stage出现变慢：

   去查看是否部分task执行时间远大于其他的task运行时间，并且处理的数据量也远大于其他的数据量。如果是就证明出现了数据倾斜，我们可以通过yarn日志来定位sql解决问题。

9. 介绍一下spark的组成和任务执行流程。

   spark的组成：master，worker，driver，executor，applicationmaster。

   以yarn-client举例：

   1. Driver向yarn的RM来申请启动一个AM。
   2. AM向RM申请contain资源，用户后续Executor的启动；RM向NM启动container。

3. AM向启动了container的NM发起请求来启动executor。
4. Driver的SparkContext分配Task给各个Executor执行，同时监控executor的运行状态。
5. 应用程序结束后，AM向RM申请注销。

与Yarn-cluster唯一的区别就是Driver的位置不同，cluster的位置位于集群内部而不是客户端本地。

10. 说一下spark中常见的调优方式。

在spark中，除了sql调优和参数调优，我们可以开启一个AQE（自适应查询执行）

主要有三个作用：

1. 合并shuffle小分区
2. sort-merge-join转化成Broadcastjoin
3. skew join优化

11. hive / spark出现任务报错应该如何排查?

首先查看调度平台，检查是否能够直接排查出问题，比如上游表是否出现变化导致任务失败；如果不是，我们可以通过yarn日志来查看具体的报错是什么。其中hive是有多个Application，但是spark只会产生一个Application。

Hive任务报错：直接根据报错信息找到对应的application，然后找到是哪个stage出现了问题。之后通过explain来查看sql执行计划，通过过滤条件，聚合字段等来定位具体的sql。

Spark任务报错：查看整个application的DAG图，然后找到报错的stageDAG图，通过报错信息DAG图中的信息在整个DAG中查找到对应位置。然后再通过整体DAG图的执行计划来对应具体的sql。

12. 说一下Hive和Spark的区别（本质就是MR和Spark的区别：四点）

Spark的落盘次数更少：Spark只在产生宽依赖的时候才会产生落盘操作；每一个MR任务都会落盘。

Spark的Persist操作：可以将常用的数据进行持久化操作。

Spark的ByPass操作：可以选择只在特殊的场景下排序。

Spark是线程级别的：线程级别的任务可以减少任务拉起消耗的资源，可以共享同一个进程中的资源。

13. 你们公司的分层架构是什么样的，谈谈你对数据分层的理解。

我们公司分了采用原始的五层架构，其中包含ODS层，DWD层，DWS层，ADS层和DIM层。提高复用性，降低维护成本，平衡存储和查询性能。

ODS层：将数据原封不动的迁移过来，方便进行追溯问题。

DWD层：在ODS层的基础上进行去重，空值处理，统一编码等数据处理，形成可信的明细数据。

DWS层：在DWD层的基础上进行轻度汇总，用空间来换取时间，提高复用性。

ADS层：在DWS层的基础上进行重度汇总来满足具体业务的需求。

DIM层：聚集常用的维度表。

14. 介绍一下维度建模的过程，结合你的实际生产案例来解释。

以授信审批明细表为例：

确定业务过程：确定主要实体（客户，产品）

声明粒度：表中每一行的含义。在授信审批中就是用户的一次申请结果。

确定维度：如渠道维度，日期维度，授信成功状态。

确定事实：申请额度，审批额度。

15. 数据模型设计的基本原则?
    1. 高内聚低耦合：将相同业务过程放在同一个主题域中。
    2. 核心模型和边缘模型分离：将核心字段放在一个表中，其他不常用的字段放在另外一张表中。
    3. 公共处理逻辑下沉：将ADS层需要用到的公共指标下沉到公共层进行公用。
    4. 保证口径的一致性。
    5. 保证命名清晰。
16. 数据治理体系（What，Why，How）
    1. 数据治理是什么?

       数据治理是指对整个数据资产的生命周期进行管理，监督和控制的机制。比如制定数据标准，提升数据质量，保障数据安全等。
    2. 为什么要做数据治理（好处）?

       数据治理主要有以下几个作用：
       - 改善数据质量
       - 控制数据风险
       - 增强数据安全
       - 赋能业务决策
    3. 数据治理从哪几个方面做?

       数据标准管理：统一数据定义、编码和口径。比如通过建设数据字典和指标管理体系来规定指标的命名规范和开发口径。

       元数据治理：
17. 如何保证指标的一致性?

    指标的一致性分为口径一致性和命名一致性：

    口径一致性：先查询公司统一维护的指标字典是否存在相关的加工逻辑，如果没有再根据需求创建并维护到平台中。

    命名一致性：按照公司统一的命名规范。

    表命名规范：层级+数据域+业务过程+分区类型
18. 你在公司里面配置过哪些DQC规则，常见的DQC分为哪几类?

    常见的报警规则包括是否为空，是否唯一，是否在枚举值内?

    根据DQC报警程度我们可以分为不同级别的报警：

    P0级别：这个级别的我们会直接终止任务，并且直接通过电话告警。比如主键出现空值。

    P1级别：这个级别的内容我们不会直接终止任务，但是会通过邮箱来提醒负责人出现异常。比如表行数波动超过50%。

    P2级别：这个级别的内容我们会直接写入日志中。比如具体授信额度出现异常。

    拓展：

    数据质量的四个方面：准确性，完整性，一致性，及时性。
19. 如果上游表出现延迟，导致下游多个依赖的表出现SLA面临风险，你会怎么办?

    首先评估影响范围：哪些下游表受到了影响，它们的SLA是否可以顺延?

    如果不可以，提前通知相关业务方，告知他们我们预计产出时间和补偿方案。

    事后进行复盘：是sql出现了问题还是上游表出现问题

20. 举一个你自己创建Skills来提高开发效率的案例。

    在我们自研的数据开发平台中，我们集成了一个agent聊天助手，然后也可以通过自己上传skill来使用。比如我之前自己创建了一个sql和需求文档对齐的一个skill，这个skill需要我上传一个写好的需求文档，然后我将写好的sql给它，agent通过skill来完成sql需求对齐功能。

    在后续的迭代更新中，主要从以下三点来进行优化：

    1. 细化工作流：分别提取开发所需的指标和维度；提取sql逻辑；识别sql所需的粒度级别等。
    2. 我还在skill中添加了很多常见的报错案例来供agent进行参考，比如统计UV时没有使用distinct等常见的问题。
    3. 我通过设计输出模板来使得输出物更加易懂，比如：结论；核心问题；需求覆盖度；修改建议等。

21. 你们公司在测试环节通过哪几个方面来实现对sql代码的测试？

    我们公司主要是在代码测试之前来对sql代码进行一个测评。我们公司打造了一个案例知识库，里面汇总了常见的一些错误案例，比如说时间处理的案例，提前进行数据过滤的案例来供agent进行参考，然后集成在数据开发平台界面中的一个小按钮，我们可以通过点击这个按钮来实现对sql的一键式校验。

    但是我看一些大厂公司也会在对sql进行动态测试，比如通过传入hive的元数据库信息，然后通过这些信息来查看选取的源头表是否正确，指标口径是否满足需求。

22. 是怎么通过自研的平台来实现ODS层的同步？（你们公司使用的技术栈有哪些？）

    我们公司有自己的自研开发平台，在实现ODS层同步的时候我们一般使用自研平台中的Datax，只需要制定元数据表和目标数据表即可。

    我们公司的主要技术栈是：hive+spark用于计算，调度平台是小海豚，数据探查平台是presto，业务数据库是oracle，

23. 说一下你们公司的需求承接整体流程

    1. 提出需求：业务方提出需求。
    2. 需求分析：由产品经理来进行需求分析，判断这个需求是否合理。
    3. 需求承接：产品经理将这个需求放在jira上，其中会包括需求背景，需求内容，产出物是什么等。
    4. 需求评审：如果是简单的需求的话，我们会直接和产品经理进行对接，然后直接进行开发；如果是比较复杂的需求的话，我们会拉上产品尽力，业务方和数仓负责人等开一个会议，来确定需求的可行性，指标的口径和需求排期等。
    5. 模型开发：
        - 表设计开发：设计表名，字段（维度和指标）；外键和主键；注释；表分区等。
        - map文档设计：每个字段的生成逻辑，表与表之间的关系等。

          上面两个表会进行归档，方便之后来检查复用。
        - 调度设计：通过上下游关系来设计调度任务。
    6. 自测：主要从过滤条件；口径指标的计算是否准确；sql是否可以优化等角度来进行自测。
    7. 数据验收
    8. 上线

24. 说一下需求文档的基本内容有哪些？

    需求背景及价值，需求内容，所需的产出物是什么？

25. 简单介绍一下指标开发和报表开发的实际用例？

以授信通过率为例：

ODS层：订单申请表

DIM层：渠道表和用户信息表

DWD层：将订单信息表和渠道表和用户信息表进行关联，同时进行数据清洗处理。
DWS层：根据日期和渠道维度，统计最小授信额度，最大授信额度，平均授信额度，授信通过率和平均授信额度。

ADS层：只取我们需要的授信申请人数，通过人数，授信通过率和平均授信额度。

26. 说说你是怎么定位问题，并且对任务进行调优的？

    S：在一天早上发现渠道风险报表出现了SLA报警。

    T：业务方那边要求我们去排查问题并且要求我们能够在规定的SLA时间内完成避免下游表无法及时产出。

    A：首先我通过查看调度平台上任务的历史版本信息，发现版本信息没有发生变动。然后我去查看具体报错的application，发现报错的application中有一部分的task执行时间非常长，并且shuffle的数据量也远大于其他的task。这个时候我基本判定出现了数据倾斜原因。我通过数据探查group by字段中的渠道字段时，发现这一天的unkonwn字段数据量远大于其他的数据量。我第一时间想到的是能不能先和业务方那边沟通将这个unknown字段进行具体的划分，在得知无法进行划分时，我通过对渠道字段进行加盐处理来解决数据倾斜问题。

    R：通过加盐处理之后，成功使得这个任务能够在规定的SLA内完成，并且在之后的时间里，这个任务也没有发生SLA报警了。

27. 简单介绍一下你们公司基于Agent的数据开发智能化工具，说一下整体的流程是什么？中间遇到了哪些问题（引申出之后的迭代更新）？

    我们将工具集成在日常的数据开发平台中，用户可以上传对应的需求文档来实现自动生成SQL。主要的流程分为以下几个方面：

    需求分析：在这个步骤中，我们通过设计提示词和提供案例模板，来引导大模型总结出需求文档中的维度和指标，并且要求以固定的模板进行输出。

    知识召回：我们使用多路召回，通过配置两个知识库，其中分别是hive元数据库和业务知识数据库（常见的业务开发案例）。其中hive元数据库的作用是能够召回多个符合维度和指标的表；业务知识数据库的作用是能够根据维度和指标来召回对应的开发案例，然后抽取出其中的sql和指标开发口径。然后结合指标开发口径在之前多个召回的表中挑选出最优的表。

    生成SQL：在得到维度、指标开发口径和所需的表之后，让大模型来生成SQL并且在测试环境中使用小样本数据来进行测试，查看逻辑是否出现错误。

    拓展：你们在这个过程中遇到了哪些问题？

    1. 业务知识库迭代更新。

       S：首先第一个大的问题就是在配置知识库时召回不准确。在原始的版本中，我们将业务知识库和hive元数据库放在一起，但是这会导致我们召回的内容不符合我们的预期，无法召回最优的表或者指标加工口径。

       T：我们需要想办法来优化我们召回的效果。

       A：为了解决这个问题，我们将两个知识库进行分开处理，其中hive数据库专门负责根据之前的维度和指标来选择召回数量较多的表；业务知识数据库则负责根据召回的业务知识来提取sql和指标口径。最后再结合这两者来筛选出最终的表。

       R：通过这种方式，我们能够更有针对性的去召回对应的内容。

    2. 知识库内容的优化

S：在我们对知识库召回内容进行测试的时候，我们发现召回的内容很多都是不完整的内容，比如案例不完整导致无法获取正确的sql加工逻辑。

T：我们需要想办法能够让知识库切分完整的内容。

A：我们根据不同案例层级分类和统一归类。我们将内容进行分段处理，然后将同一指标体系的内容放在一起能够保证获取的sql是完整的。

R：通过这种方式能够保证我们召回的内容是完整的，不会出现被截断的sql加工逻辑导致任务失败。

通过以上两种优化手段成功将测试通过率从60%提高至80%+。